

Ressource R2.02 : Développement d'application avec IHM

JavaFX - Séance TP 1 : gestion des événements

Les premières séances de TD ont permis de créer le contenu des fenêtres par l'ajout de composants graphiques. On va maintenant rendre ces fenêtres réactives à des événements tels un clic de souris, l'appui sur une touche du clavier, la sélection d'un élément dans une liste, la fermeture d'une fenêtre, etc.

Cette séance présente le principe de gestion des événements puis propose de programmer quelques exemples.

1. Programmation événementielle

Dans la programmation impérative (séquence d'instructions), c'est le programme qui dirige les opérations. Avec la **programmation événementielle**, ce sont les **événements** qui pilotent l'application. Ils sont généralement déclenchés par l'utilisateur, mais peuvent l'être aussi par le programme lui-même. Ce mode non directif convient bien à la gestion des interfaces graphiques. Dans la programmation événementielle, un processus surveille constamment les actions de l'utilisateur susceptibles de déclencher des événements qui peuvent être ensuite traités par le programme.

2. Programmer un événement

En JavaFX, les événements sont représentés par la classe `Event` ou par ses sous-classes (voir la documentation sur le package `javafx.event`).

Un événement se caractérise notamment par :

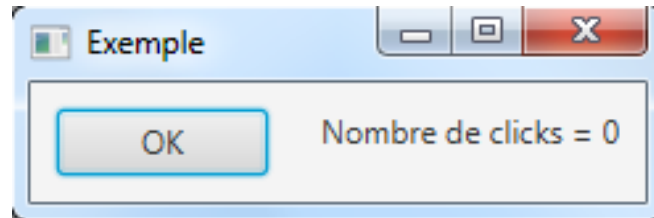
- son **type** (par exemple `KeyEvent`, `MouseEvent`, etc. qui sont des sous-classes de la classe `Event`),
- sa **source** (composant qui est à l'origine de l'événement),
- et éventuellement sa **cible**.

Programmer un événement c'est à la fois :

- **détecter** l'événement : la détection d'un événement sur un composant (par exemple un clic sur un bouton) se fait en "posant" sur le composant un détecteur d'événement, généralement appelé un écouteur (*listener* en anglais),
- et **traiter** l'événement : le traitement de l'événement consiste à écrire une méthode java correspondant à la réaction souhaitée (on appelle ça une "*callback*").

3. Mise en œuvre possible

Pour la programmation des événements, plusieurs techniques sont possibles, mais nous allons en aborder une simple et pratique. Considérons cette IHM (rudimentaire). À chaque clic sur le bouton OK, le message doit s'actualiser pour montrer le nombre de clics déjà effectués.



➤ *Programmez cette fenêtre. Elle est composée d'un Button et d'un Label situés dans une HBox. La classe contient une variable de type `int` pour compter le nombre de clics.*

```
public class Fenetre extends Application {
    private int    nbClics    = 0;
    private HBox   root      = new HBox();
    private Button boutonOK   = new Button("OK");
    private Label  texte      = new Label("Nombre de clics = " + nbClics);

    ETC.
    ...

    boutonOK.setPrefWidth(80);
    root.setSpacing(20);
    root.setPadding(new Insets(10));
    root.getChildren().addAll(boutonOK, texte);
    Scene laScene = new Scene(root);
    this.setScene(laScene);
    this.setTitle("Exemple");
    this.show();

    ETC.
    ...
}
```

Gestion de l'événement par une expression lambda

Gérer un événement consiste ici à :

- utiliser des **méthodes utilitaires** propres à certains composants pour la pose des écouteurs,
- et à utiliser une **expression lambda** pour implémenter la callback (=programmer le gestionnaire d'événement).

Une expression lambda est une méthode anonyme : sa définition se fait sans déclaration explicite du type de retour, ni de modificateurs d'accès, ni de nom. C'est un raccourci syntaxique qui permet de définir une méthode directement à l'endroit où elle est utilisée.

Une expression lambda prend la forme suivante :

paramètres $\rightarrow$ { *traitements* ; }

Revenons à notre exemple. Dans le constructeur de la fenêtre, on va utiliser la méthode `setOnAction(...)` pour poser l'écouteur et lui passer en paramètre une expression lambda :

```
boutonOK.setOnAction( e -> {  
    nbClics++;  
    texte.setText("Nombre de clics = "+ nbClics );  
});
```

Ou bien encore (méthode à privilégier !) :

Pose de l'écouteur.

```
boutonOK.setOnAction( e -> incrementerNbClics() );  
  
...  
private void incrementerNbClics(){  
    nbClics++;  
    texte.setText("Nombre de clics = "+ nbClics);  
}
```

Méthode qui traite
l'événement (la callback).

À FAIRE

Programmez l'événement "clic" sur le bouton OK de cette fenêtre.

4. Remarques

4.1. Un composant, plusieurs gestionnaires d'événement

Un composant peut en général subir plusieurs types d'événement. Il dispose alors des méthodes utilitaires correspondantes de type **setOn...()**. Par exemple, un composant réceptif à un clic de souris, à une frappe de caractère, au début d'un glisser/déposer, etc. fera appel aux méthodes `setOnAction(...)`, `setOnKeyTyped(...)`, `setOnDragDetected(...)`, etc.

Consultez la documentation pour découvrir ces méthodes.

4.2. Plusieurs composants, un même gestionnaire d'événement

Par ailleurs, un même événement peut concerner plusieurs composants. C'est le cas des boutons d'une fenêtre qui doivent répondre à un clic de souris. Ils peuvent faire appel à la même gestionnaire d'événement.

```
bnAnnuler.setOnAction(e -> {gererClic(e);});
```

```
bnOK.setOnAction(e -> {gererClic(e);});
```

Les deux boutons ont le même gestionnaire d'événement. L'événement `e` est passé en paramètre.

Le gestionnaire d'événement est quand même capable de savoir quel composant a subi l'événement, grâce à la méthode `getSource()` qui renvoie l'élément "source" de l'événement.

```
private void gererClic(ActionEvent e) {  
    if (e.getSource()==bnOK){  
        // Faire une action  
    } else if (e.getSource()==bnAnnuler){  
        // Faire une autre action  
    }  
}
```

`getSource()` permet de savoir quel bouton a été cliqué.

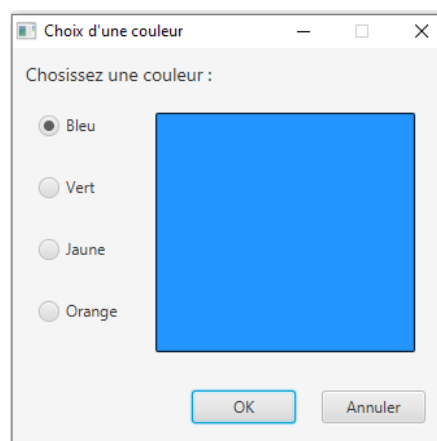
5. Exercices

Important

Pour les exercices qui suivent, les classes java correspondant aux fenêtres sont disponibles sur Moodle. Vous devez juste les compléter pour programmer les événements en suivant les explications fournies.

5.1. Le choix de couleur

Programmez le comportement de cette fenêtre qui doit permettre de choisir une couleur à l'aide des boutons radios, puis d'enregistrer son choix en cliquant sur OK. Bien entendu, tout changement de sélection de bouton radio entraîne la modification de la couleur du rectangle. Les événements à prendre en compte ici sont tous de type `ActionEvent` : clic sur les boutons radios et clic sur les boutons OK et Annuler.



Comportement attendu :

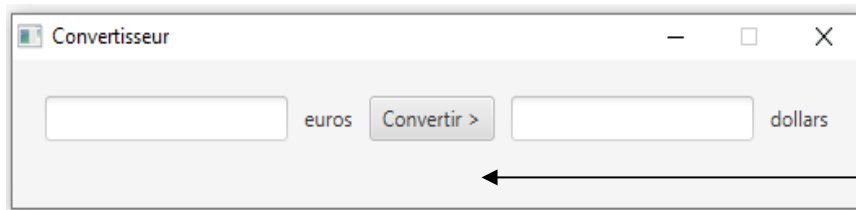
- lors d'un clic sur un bouton radio, le rectangle prend la couleur correspondante (utilisez la méthode `setFill(...)`),
- un clic sur **OK** affiche un message dans la console, par exemple :
`Couleur choisie : BLEU`
- un clic sur **Annuler** ferme la fenêtre (utilisez la méthode `close()`).

Pour aller plus loin...

Complétez votre programme en faisant en sorte que le rectangle change de couleur quand l'utilisateur survole avec la souris un bouton radio. Dès que la souris quitte le bouton radio, la couleur sélectionnée est rétablie dans le rectangle.

5.2. Le convertisseur euros/dollars

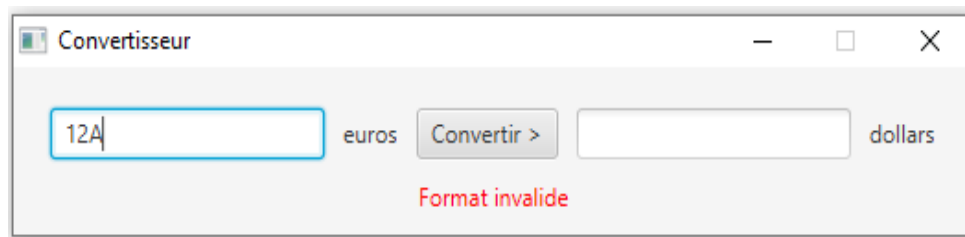
Le but de cet exercice est de réaliser un convertisseur *euros* ➔ *dollars* dans une fenêtre qui se présente comme suit :



zone de message
(un Label, invisible
par défaut)

La saisie du montant en euros

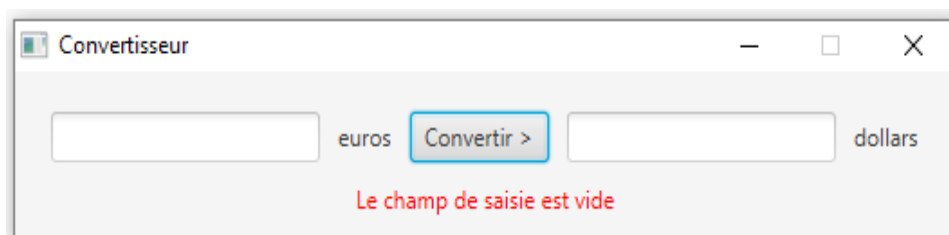
Il faudra signaler la frappe d'un caractère non autorisé et afficher un message d'erreur :



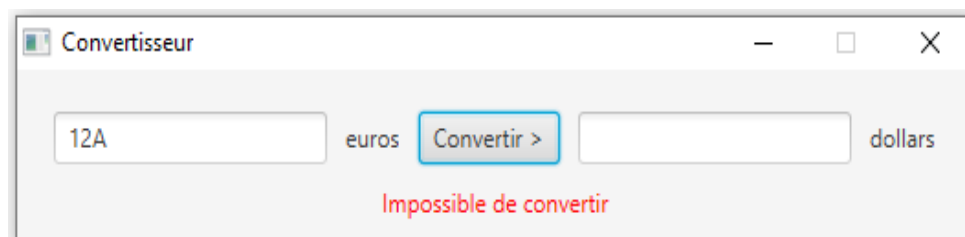
Il faudra détecter et traiter un événement de type `KeyEvent`. Utilisez la méthode `setOnKeyReleased(...)` pour traiter la frappe d'une touche du clavier. La méthode `getCode()` permet de savoir quelle touche a été frappée.

Le bouton Convertir

Un click sur ce bouton déclenchera le calcul et l'affichage du résultat dans le champ de droite (le taux de change est une constante). Si le champ de gauche est vide, alors un message d'erreur doit apparaître :



Et si le champ ne correspond pas à un nombre, un autre message doit être affiché :



Remarque : pour tester la validité du montant en euros, utilisez la méthode `isNumeric()` fournie dans la classe `FenConvertisseur`.

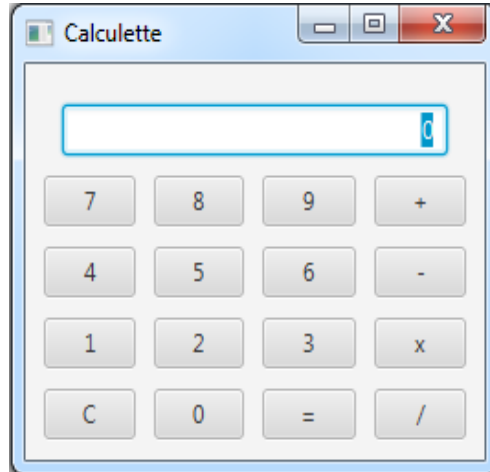
Pour tester votre programme...

Cette matrice présente les résultats attendus en fonction des événements prévus et de l'état de l'IHM. Une fois votre programme terminé, testez systématiquement chacun de ces cas et regardez si ce que vous obtenez est conforme à ce qui est attendu.

Évts États	Frappe d'une touche autorisée	Frappe d'une touche non autorisée	Clic sur Convertir
Zone de saisie vide	<i>Pas de message d'erreur</i> <i>Zone de saisie actualisée</i> <i>Zone de résultat = Vide</i>	<i>Message d'erreur = "Format invalid"</i> <i>Zone de saisie actualisée</i> <i>Zone de résultat = Vide</i>	<i>Message d'erreur = "Le champ de saisie est vide"</i> <i>Zone de saisie inchangée</i> <i>Zone de résultat = Vide</i>
Valeur saisie numérique	<i>Pas de message d'erreur</i> <i>Zone de saisie actualisée</i> <i>Zone de résultat = Vide</i>	<i>Message d'erreur = "Format invalid"</i> <i>Zone de saisie actualisée</i> <i>Zone de résultat = Vide</i>	<i>Pas de message d'erreur</i> <i>Zone de saisie inchangée</i> <i>Zone de résultat = la valeur convertie en dollars</i>
Valeur saisie non numérique	<i>Pas de message d'erreur</i> <i>Zone de saisie actualisée</i> <i>Zone de résultat = Vide</i>	<i>Message d'erreur = "Format invalid"</i> <i>Zone de saisie actualisée</i> <i>Zone de résultat = Vide</i>	<i>Message d'erreur = "Impossible de convertir"</i> <i>Zone de saisie inchangée</i> <i>Zone de résultat = Vide</i>

5.1. La calculette

On se propose de programmer cette calculette dont le fonctionnement est (très) simplifié : elle effectue des opérations simples entre deux nombres : l'utilisateur "tape" d'abord une suite de chiffres pour le premier nombre, clique ensuite sur l'un des quatre opérateurs, tape une autre série de chiffres pour le deuxième nombre et enfin clique sur "=".



- 1) Lorsque l'utilisateur appuie sur la touche d'un chiffre, c'est ce chiffre qui s'affiche la première fois, puis ensuite c'est la concaténation des chiffres.
- 2) Lorsqu'il appuie sur la touche d'un opérateur (+, -, * ou /), il faut mémoriser cet opérateur, l'affichage ne change pas mais il faut se préparer à la saisie du deuxième nombre.
- 3) Lorsqu'il appuie sur la touche « = », il faut calculer le résultat et l'afficher.

Si vous avez le temps, programmez le clic sur le bouton C.

Tests du programme.

À la manière de l'exercice 2, essayez de tester de manière systématique toutes les situations possibles (bien sûr, vous n'oublierez pas le cas particulier de la division par 0)